# TASLEEM BANU Y M

## ANALYSIS OF CYBERSECURITY ATTACKS

### 1. Cybersecurity Threats Dataset – Abstract

This project analyzes cybersecurity attack data to identify common cyber threats, recognize suspicious network activity, and understand how security systems respond to incidents. The analysis covers attack categories such as Malware, DDoS, and Intrusion and uses network traffic, security logs, anomaly scores, severity levels, and response actions to identify useful patterns. The overall purpose is to support faster threat detection, improve network security, and reduce cybersecurity risk.

### 2. Cybersecurity Threat Detection – Problem Overview

The rapid growth of computer networks and digital services has increased the volume and complexity of cybersecurity threats. Monitoring thousands of network records manually is difficult and time-consuming. This project therefore applies data analysis and preprocessing techniques to examine cybersecurity incidents, identify attack types, understand their severity, and study the actions taken against suspicious activity.

### 3. Cybersecurity Attacks Dataset – Overview

| | |
|---|---|
| Team | 1 |
| Dataset | Cybersecurity Attacks Dataset |
| Theme | Crime and Public Safety |
| Topic | Cybersecurity Threats |
| Size | 1,499 rows × 25 columns |

The dataset contains information about network activities and cybersecurity incidents, including source and destination addresses, ports, protocols, packet details, traffic characteristics, malware indicators, anomaly scores, alerts, attack types, attack signatures, actions taken, severity levels, firewall logs, and IDS/IPS alerts.

### 4. Cybersecurity Network & Attack Attributes

| Attribute | Description |
|---|---|
| Timestamp | Date and time of the network activity or attack. |
| Source IP Address | IP address of the device initiating the network connection. |
| Destination IP Address | IP address of the target device receiving network traffic. |
| Source Port | Port used by the source device to send traffic. |
| Destination Port | Port on the destination system receiving traffic. |

| | |
|---|---|
| Protocol | Network communication protocol, such as TCP, UDP, or ICMP. |
| Packet Length | Size of the transmitted network packet. |
| Packet Type | Category of packet, such as Data or Control. |
| Traffic Type | Nature/type of network traffic. |
| Payload Data | Actual content carried by a network packet. |
| Malware Indicators | Indicators suggesting malware-related activity. |
| Anomaly Scores | Numerical measure of how unusual or suspicious activity is. |
| Alerts/Warnings | Security alerts or warnings generated by monitoring systems. |
| Attack Type | Detected cyberattack category, such as Malware, DDoS, or Intrusion. |
| Attack Signature | Known pattern/signature associated with an attack. |
| Action Taken | Security response such as Blocked, Allowed, Logged, or Ignored. |
| Severity Level | Threat seriousness, such as Low, Medium, or High. |
| User Information | Information related to the user/account involved. |
| Device Information | Details about the device or operating system. |
| Network Segment | Network section in which the activity occurred. |
| Geo-location Data | Geographic information associated with the activity/IP. |
| Proxy Information | Proxy-server information, where applicable. |
| Firewall Logs | Records generated by the firewall. |
| IDS/IPS Alerts | Alerts generated by intrusion detection/prevention systems. |
| Log Source | System or security tool that generated the event/log. |

## 5. Initial Inspection of Cybersecurity Attack Data

**Display first five records:** Shows the first five rows and gives an initial view of the dataset.

```
df.head()
```

**Display last five records:** Shows the final five rows and helps inspect the ending records.

```
df.tail()
```

**Dataset information:** Displays rows, columns, column names, data types, non-null information, and memory details.

```
df.info()
```

**Statistical summary:** Provides count, mean, standard deviation, minimum, maximum, and quartiles for numerical data.

```
df.describe()
```

**Column names:** Displays all column names.

```
df.columns
```

**Dataset shape:** Returns the number of rows and columns.

```
df.shape
```

## 6. Duplicate Cybersecurity Incident Records

Duplicate records should be checked before analysis so that repeated observations do not distort results.

```
df.duplicated().sum()
```

The duplicate check in the project showed no duplicate records, so no rows needed to be removed.

```
df.drop_duplicates(inplace=True)
```

## 7. Missing Values in Cybersecurity Data

Missing values were examined using the following code:

```
df.isnull().sum()
```

The project materials report missing values mainly in Malware Indicators, Alerts/Warnings, Proxy Information, Firewall Logs, and IDS/IPS Alerts. The first project document additionally identifies one missing value in an 'Unnamed: 1' column. The reported counts differ slightly between the two source documents, so the exact counts should be taken from the actual dataset/code output rather than assuming the two reports are identical.

| Column | Reported missing values |
|---|---|
| Malware Indicators | 756–757 |
| Alerts/Warnings | 762–763 |
| Proxy Information | 750–751 |
| Firewall Logs | 766 |
| IDS/IPS Alerts | 750–751 |
| Unnamed: 1 | 1 (reported in one source) |

## 8. Identifying Incomplete Cybersecurity Attributes

```
null_values = df.isnull().sum()
null_values[null_values > 0]
```

This first calculates missing values for every column and then displays only columns whose missing-value count is greater than zero.

## 9. Attack Type and Severity Category Analysis

Before applying condition-based preprocessing, the unique values in the important categorical fields are inspected.

```
df['Attack Type'].unique()
df['Severity Level'].unique()
```

These checks help identify the actual attack and severity categories present in the dataset before writing logical rules.

## 10. Cybersecurity Data Cleaning and Missing-Value Treatment

### 10.1 Malware Indicator Data Cleaning

A domain-based treatment is used so that missing malware information can be interpreted using related fields. When the value is missing, the row is checked against Attack Type and Severity Level; otherwise, the existing value is retained.

```
def fill_malware_indicator(row):
    if pd.isnull(row['Malware Indicators']):
        if str(row['Attack Type']).lower() == 'malware':
            return 'Malware Detected'
        elif str(row['Severity Level']).lower() == 'high':
            return 'Potential Malware'
        else:
            return 'No Specific Indicator'
    else:
        return row['Malware Indicators']

df['Malware Indicators'] = df.apply(fill_malware_indicator, axis=1)
df['Malware Indicators'].isnull().sum()
```

### 10.2 Proxy Information Data Cleaning

When proxy information cannot be determined from the available data, it is represented as Unknown rather than inventing a value.

```
df['Proxy Information'] = df['Proxy Information'].fillna('Unknown')
df['Proxy Information'].isnull().sum()
```

### 10.3 Firewall Log Data Cleaning

Missing firewall-log entries are represented as No Firewall Log when the dataset meaning supports interpreting a missing record as no recorded firewall log.

```
df['Firewall Logs'] = df['Firewall Logs'].fillna('No Firewall Log')
df['Firewall Logs'].isnull().sum()
```

### 10.4 Security Alerts and Warnings Data Cleaning

Missing alerts are handled using severity and response information. High-severity rows can receive High Severity Alert; blocked threats can receive Threat Blocked; otherwise the value becomes No Alert Recorded.

```
def fill_alerts(row):
    if pd.isnull(row['Alerts/Warnings']):
        if str(row['Severity Level']).lower() == 'high':
            return 'High Severity Alert'
        elif str(row['Action Taken']).lower() == 'blocked':
            return 'Threat Blocked'
        else:
            return 'No Alert Recorded'
    else:
        return row['Alerts/Warnings']

df['Alerts/Warnings'] = df.apply(fill_alerts, axis=1)
df['Alerts/Warnings'].isnull().sum()
```

## 11. Final Cybersecurity Dataset Validation

After preprocessing, the dataset is checked again to confirm the result of the cleaning process.

```
df.isnull().sum()
df.duplicated().sum()
df.head()
```

The final null-value check identifies any remaining missing values, the duplicate check confirms whether repeated rows remain, and df.head() provides a visual inspection of the cleaned dataset.

## 12. Cybersecurity Threat Analysis – Conclusion

The combined analysis presents a structured workflow for studying cybersecurity threats: understand the dataset, inspect its structure, check duplicates, identify missing values, inspect important categories, apply meaningful missing-value rules, and validate the cleaned data. The approach preserves existing information where available and uses contextual rules for selected missing fields. The cleaned dataset can then be used for further analysis of attack types, severity levels, suspicious activity, and security responses.